



INSTITUTO FEDERAL
Sudeste de Minas Gerais

Campus
Manhuaçu

Avaliação 2 (AV2)

2 pontos

Código: INF03098
Disciplina: Engenharia de Software III
Data: _____
Professor(a): Filipe Fernandes

Orientações

1. A prova é individual e sem consulta.
2. Leia atentamente a contextualização do projeto antes de responder às questões.
3. Escreva as respostas na folha de respostas, identificando cada arquivo pelo seu caminho completo.
4. Em arquivos YAML, a indentação faz parte da sintaxe e será considerada na correção.
5. A interpretação das questões faz parte da avaliação.

Contextualização

A aplicação **Copa Resultados** é um webapp que lista todas as partidas da Copa do Mundo de 2026, da fase de grupos até a final. Os dados são obtidos do projeto aberto `openfootball/worldcup.json` e armazenados no banco `cupmatches`. A aplicação é composta por três partes:

- **Backend (api/)**: API em Node.js, TypeScript, Express e Mongoose, com os endpoints `GET /matches` (retorna o torneio com todas as partidas) e `GET /health` (verificação de saúde). Possui script de *seed* que baixa o calendário oficial e popula o banco, e testes BDD com Cucumber.
- **Frontend (web/)**: aplicação em Vite, TypeScript e React que consome a API e exibe as partidas. Em produção, os arquivos estáticos são servidos pelo Nginx.
- **Banco de dados**: MongoDB.



Figura 1: Tela principal do webapp Copa Resultados.

Estrutura do projeto

O projeto está pronto e funcional, mas **ainda não está containerizado**. A estrutura de pastas é a seguinte (os arquivos marcados são os que você deverá escrever nesta prova):

```
1 App/
2 |-- .github/
3 |   |-- workflows/
4 |     |-- api-test-bdd.yml # <== QUESTAO 4
5 |-- api/                  # Backend (Node.js + TypeScript + Express + Mongoose)
6 |   |-- src/              # config/, controllers/, models/, routes/, seed.ts,
   |   server.ts
7 |   |-- features/         # Testes BDD (Cucumber)
8 |   |-- .env.dev          # Variaveis de ambiente (execucao local)
9 |   |-- Dockerfile        # <== QUESTAO 1
```

```
10 |-- web/                # Frontend (Vite + React + TypeScript)
11 |   |-- src/            # components/, api.ts, App.tsx, main.tsx
12 |   |-- .env.dev        # Variaveis de ambiente (execucao local)
13 |   |-- nginx.conf      # Configuracao do Nginx (ja existe, pronta para uso)
14 |   |-- Dockerfile      # Ja existe, pronto para uso (nao e cobrado)
15 |-- docker-compose.yml  # <== QUESTAO 2
16 |-- .env                # <== QUESTAO 3
17 |-- README.md
```

Detalhes importantes do projeto

- **Imagens:** Node 20 (node:20-alpine), Nginx (nginx:alpine) e MongoDB 7 (mongo:7).
- **Portas internas:** a API escuta na porta 3000; o Nginx do container web escuta na porta 80; o MongoDB usa a porta 27017.
- **Rede do Docker Compose:** dentro da rede, a API acessa o banco pelo **nome do serviço** (mongodb), e não por localhost.
- **Build do frontend:** a variável VITE_API_URL é usada em **tempo de build** do Vite; por isso deve ser passada como *build arg* para o Dockerfile do web.
- **Nginx:** o arquivo web/nginx.conf já existe e deve ser copiado para /etc/nginx/conf.d/default.conf; o build do Vite gera a pasta dist, que deve ser copiada para /usr/share/nginx/html.
- **Scripts npm do backend (api/package.json):**
 - npm run dev — executa a API em modo de desenvolvimento, com recarga automática.
 - npm run build — compila o TypeScript para dist/.
 - npm start — executa a API já compilada (node dist/server.js).
 - npm run seed — baixa o calendário oficial e popula o banco.
 - npm run test:bdd — executa os testes BDD com Cucumber.
- **Variáveis de ambiente da raiz** (usadas pelo Docker Compose), com os valores exatos do projeto:

```
1 API_PORT=3000
2 WEB_PORT=5173
3 MONGODB_PORT=27017
4 MONGODB_DATABASE=cupmatches
5 MONGODB_URI=mongodb://mongodb:27017/cupmatches
```

- **Execução local sem Docker:** os arquivos api/.env.dev e web/.env.dev já existem no projeto, com o seguinte conteúdo:

```
1 # api/.env.dev
2 PORT=3000
3 MONGODB_URI=mongodb://localhost:27017/cupmatches
4
5 # web/.env.dev
6 VITE_API_URL=http://localhost:3000
```

- **Integração contínua:** o repositório deve possuir o workflow de *GitHub Actions* `.github/workflows/api-test-bdd.yml`, que executa os testes BDD da API a cada *push* no repositório remoto (detalhado na Questão 4).
- **Frontend já containerizado:** o arquivo `web/Dockerfile` já existe e está pronto para uso; nesta prova você não precisa escrevê-lo.

Questões

Questão 1

0,5 ponto

Escreva o conteúdo completo do arquivo `App/api/Dockerfile`. O Dockerfile deve ser *multi-stage*, com duas etapas:

- **build:** instala todas as dependências e compila o TypeScript;
- **production:** instala somente as dependências de produção e executa a API já compilada.

Questão 2

0,5 ponto

Escreva o conteúdo completo do arquivo `App/docker-compose.yml`, orquestrando os serviços `mongodb`, `api` e `web`. Considere que:

- as portas expostas no host e as demais configurações devem usar as variáveis de ambiente do arquivo `.env` da raiz (apresentadas na contextualização);
- os dados do MongoDB devem persistir em um volume nomeado;
- o serviço `api` depende do `mongodb`, e o serviço `web` depende da `api`;

Questão 3

0,2 ponto

Com base nas variáveis de ambiente apresentadas na contextualização, escreva o conteúdo completo do arquivo `App/.env`, utilizado pelo Docker Compose.

Questão 4

0,5 ponto

Escreva o conteúdo completo do arquivo `App/.github/workflows/api-test-bdd.yml`, o workflow de *GitHub Actions* que executa os testes BDD da API a cada *push* no repositório. Considere que:

- o workflow chama-se **API - Testes BDD** e deve disparar em qualquer *push* e também manualmente pela aba *Actions* (gatilho `workflow_dispatch`);
 - há um único *job* chamado `test-bdd` (nome de exibição **API - testes BDD**) que roda em `ubuntu-latest`;
 - o *job* usa um *service container* de banco de dados com a imagem `mongo:7`, publicando a porta `27017:27017`;
 - por padrão, todos os comandos `run` devem executar no diretório `App/api` (`working-directory`);
 - o *job* define as variáveis de ambiente `MONGODB_URI=mongodb://localhost:27017/cupmatches`, `PORT=3000` e `API_URL=http://localhost:3000`;
 - os passos, nesta ordem, são:
 - *checkout* do código (`actions/checkout@v4`);
 - configurar o Node.js 20 (`actions/setup-node@v4`);
 - instalar as dependências;
 - compilar o TypeScript;
 - popular o banco com o *seed*;
 - subir a API em segundo plano e aguardar 5 segundos; e
- ```
1 run: |
2 npm start &
3 sleep 5
```
- executar os testes BDD.

Os *scripts* npm disponíveis são: `npm install`, `npm run build`, `npm run seed`, `npm start` e `npm run test:bdd`.

### Questão 5

0,3 ponto

Considere o seguinte cenário: o Docker está iniciado; todos os pacotes do projeto estão instalados; o link do repositório remoto já foi adicionado ao projeto; e todas as mudanças estão comitadas. Escreva, na ordem correta, a sequência de comandos para **subir os containers** e, em seguida, **atualizar o repositório remoto**.

## Folha de Respostas

**Avaliação:** Avaliação 2 (AV2) – 2 pontos

**Disciplina:** INF03098 – Engenharia de Software III

**Data:** \_\_\_\_\_

**Nome do(a) aluno(a):** \_\_\_\_\_

**Nota da correção:** \_\_\_\_\_

### Questão 1

- 1 \_\_\_\_\_
- 2 \_\_\_\_\_
- 3 \_\_\_\_\_
- 4 \_\_\_\_\_
- 5 \_\_\_\_\_
- 6 \_\_\_\_\_
- 7 \_\_\_\_\_
- 8 \_\_\_\_\_
- 9 \_\_\_\_\_
- 10 \_\_\_\_\_
- 11 \_\_\_\_\_
- 12 \_\_\_\_\_
- 13 \_\_\_\_\_
- 14 \_\_\_\_\_
- 15 \_\_\_\_\_
- 16 \_\_\_\_\_
- 17 \_\_\_\_\_
- 18 \_\_\_\_\_
- 19 \_\_\_\_\_
- 20 \_\_\_\_\_

**Questão 2**

**Questão 2 (continuação)**

**Questão 3**

- 1 \_\_\_\_\_
- 2 \_\_\_\_\_
- 3 \_\_\_\_\_
- 4 \_\_\_\_\_
- 5 \_\_\_\_\_
- 6 \_\_\_\_\_
- 7 \_\_\_\_\_
- 8 \_\_\_\_\_
- 9 \_\_\_\_\_
- 10 \_\_\_\_\_

**Questão 4**



**Questão 4 (continuação)**

**Questão 5**

- 1 \_\_\_\_\_
- 2 \_\_\_\_\_
- 3 \_\_\_\_\_
- 4 \_\_\_\_\_
- 5 \_\_\_\_\_
- 6 \_\_\_\_\_
- 7 \_\_\_\_\_
- 8 \_\_\_\_\_
- 9 \_\_\_\_\_
- 10 \_\_\_\_\_